

SparkFun Pulse Oximeter and Heart Rate Monitor Hookup Guide

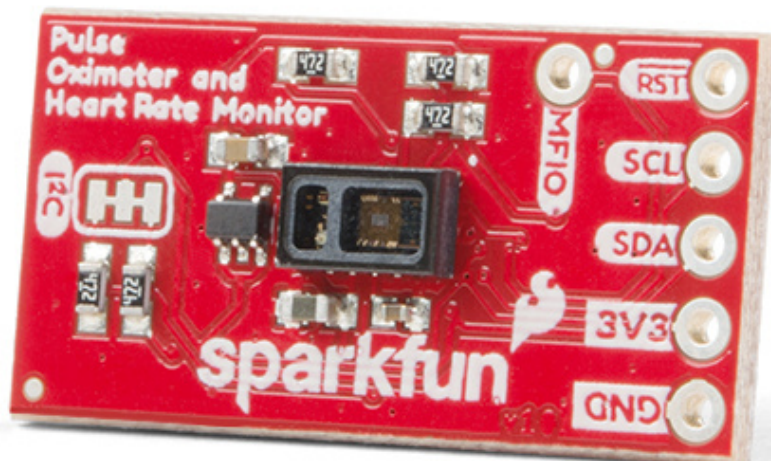
Pages

CONTRIBUTORS:  [ELIAS THE SPARKIEST](#)



Introduction

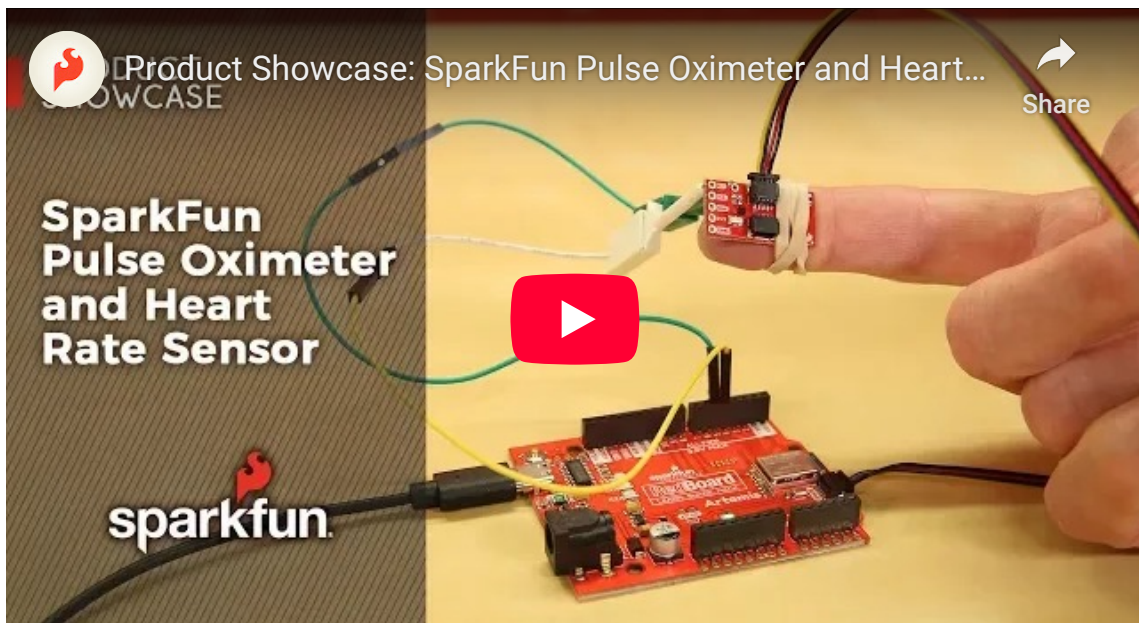
The [SparkFun Pulse Oximeter and Heart Rate Monitor](#) is an I²C based [biometric](#) sensor. Utilizing two chips from Maxim Integrated, the SparkFun Pulse Oximeter and Heart Rate Monitor has both the MAX30101 biometric sensor and MAX32664 biometric hub. While the former does all the sensing, the latter is an incredibly small and fast Cortex M4 processor that handles all of the algorithmic calculations, digital filtering, pressure/position compensation, advanced R-wave detection and automatic gain control. We've combined them and written an Arduino Library with example code demonstrating basic to advanced features to help get you started utilizing the SparkFun Pulse Oximeter and Heart Rate Monitor into your next project. Or if you're looking to put these IC's into a final product, Maxim has provided some features to get FDA approval. Let's get started!



SparkFun Pulse Oximeter and Heart Rate Sensor - MAX30101 & MAX32664 (Qwiic)

SEN-15219

\$48.95



NOTE: This device is not intended to diagnose or treat any conditions.

Required Materials

To follow along with the example code used in this tutorial, you will also need the following materials. You may not need everything though depending on what you have. Add it to your cart, read through the guide, and adjust the cart as necessary.



SparkFun RedBoard Qwiic

DEV-15123

\$21.95



Qwiic Cable - 100mm

PRT-14427

\$1.50



IC Hook with Pigtail

CAB-09741

\$5.75



USB Micro-B Cable - 6 Foot

CAB-10215

\$6.50

If you need different size Qwiic cables, we offer a kit that contains many sizes but we also carry them individually as well. Make sure to use a Qwiic cable of sufficient length for flexibility. Short lengths like the 50mm Qwiic cable can be harder to obtain sensor readings.



SparkFun Qwiic Cable Kit

KIT-15081

\$12.95



Qwiic Cable - 100mm

PRT-14427

\$1.50



Qwiic Cable - Breadboard Jumper (4-pin)

PRT-14425

\$1.50



Qwiic Cable - 500mm

PRT-14429

Retired

Suggested Reading

Our Qwiic ecosystem keeps growing and growing with a host of new Qwiic enabled micro-controllers and sensors, check [here for an overview](#).

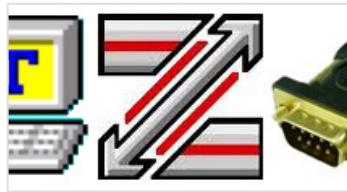


We would also recommend taking a look at the following tutorials if you aren't familiar with them.



I2C

An introduction to I2C, one of the main embedded communications protocols in use today.



Serial Terminal Basics

This tutorial will show you how to communicate with your serial devices using a variety of terminal emulator applications.



How to Work with Jumper Pads and PCB Traces

Handling PCB jumper pads and traces is an essential skill. Learn how to cut a PCB trace, add a solder jumper between pads to reroute connections, and repair a trace with the green wire method if a trace is damaged.



RedBoard Qwiic Hookup Guide

This tutorial covers the basic functionality of the RedBoard Qwiic. This tutorial also covers how to get started blinking an LED and using the Qwiic system.

Hardware Overview

Power

You can provide **3.3V** through the Qwiic connector on the "*MAX32664 Side*" of the board or through the **3V3** and **GND** labeled pins on the through hole header.

MAX30101 Side	MAX32664 Side

Qwiic Connector or I²C Pins

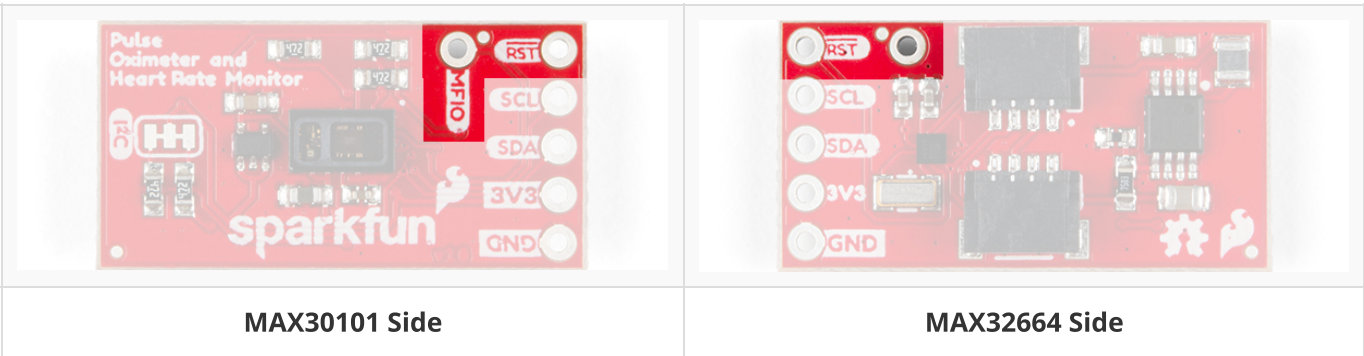
There are two Qwiic connectors on the board to easily get data from the sensor via I²C. Another option is to [solder](#) directly to the I²C plated through holes on the side of the board. Unfortunately, this board requires additional pins to function, see section below **Additional Required pins**.



We have many Qwiic sensors and Qwiic enabled microcontrollers. Check out our [Qwiic Ecosystem](#) page to get a glimpse of what else we have to offer.

Additional Required Pins

This board has two additional pins on it's header: the `RESET` and `MFIO` pin. These pins are required for the board to function because they determine if the board enters data collection mode or not. The **Hardware Hookup** section below will walk you through how to connect this board properly.



MAX30101 - Pulse Oximeter and Heart Rate Monitor

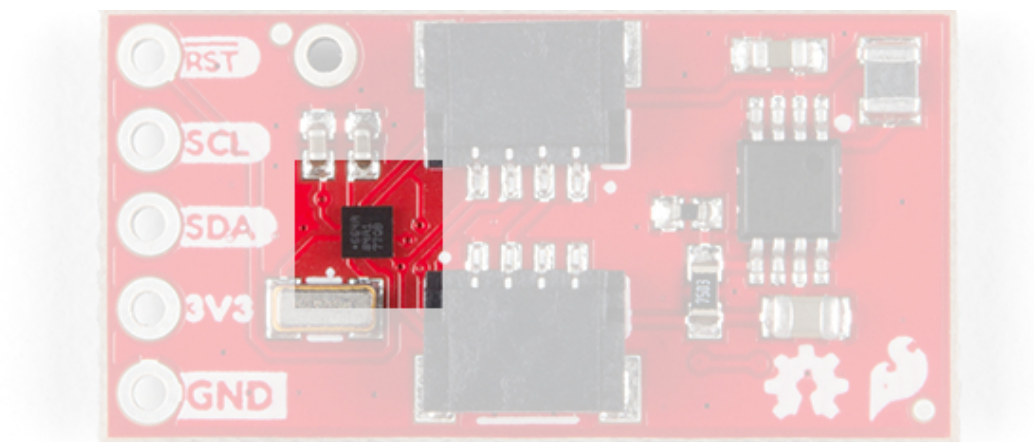
The MAX30101 gets your heart rate (BPM) and blood oxygen levels (SpO₂) through the process of [photoplethysmography](#), which is the process of obtaining the aforementioned biometric data with light. The SparkFun Pulse Oximeter works by placing your finger *gently* on the sensor in which it shines red, infrared, and sometimes green light through your skin. The capillaries filled with blood under your skin will absorb this light, or not, and the MAX30101 sensor will read which light comes back. This *light* data will then be sent back to the Biometric Sensor Hub which handles all the calculations to determine heart rate and blood oxygen levels. Simple right?!





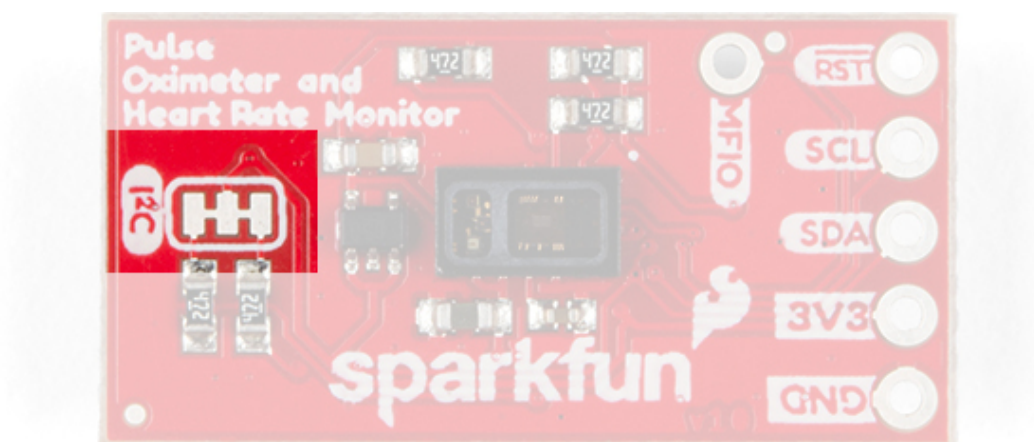
MAX32664 - Biometric Sensor Hub

The MAX32664 Biometric Sensor Hub is a very small Cortex M4 micro-controller dedicated to receiving the data it receives from the MAX30101 and running the calculations to determine heart rate and blood oxygen. When you're interfacing with the SparkFun Pulse Oximeter and Heart Rate Monitor, you are in effect interfacing with this wicked fast microcontroller. There are a multitude of settings to tailor the sensor to the persons you'll be monitoring made available through the Arduino Library we've written for it. Check the Arduino Examples below for more info!



Jumpers

There a single set of jumpers on the MAX30101 side (non Qwiic connector side) of this product. This triple jumper labeled `I2C` connects pull-up resistors to the I²C data lines. If you're daisy chaining many I²C devices together, you may need to consider [cutting these traces](#).



Dimensions

This board is very small, measuring at 1.00in x 0.5in (25.4mm x 12.7mm), which means it will fit nicely on your finger without all the bulk.



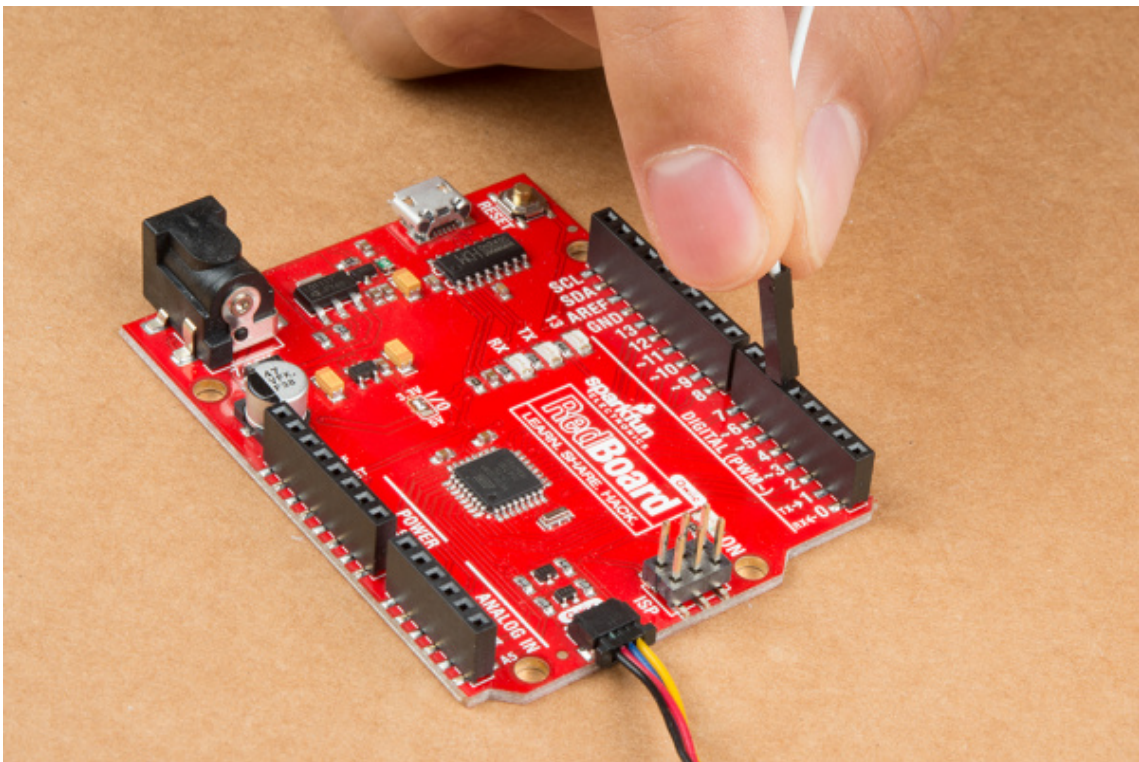


Hardware Hookup

⚡ **Warning!** Remember, the Pulse Oximeter and Heart Rate Monitor is a **3.3V sensor**. We highly recommend using a 3.3V Arduino. In this case, we [set the I/O jumper](#) to 3.3V on the RedBoard Qwiic before connecting `MFIO` and `RESET`.

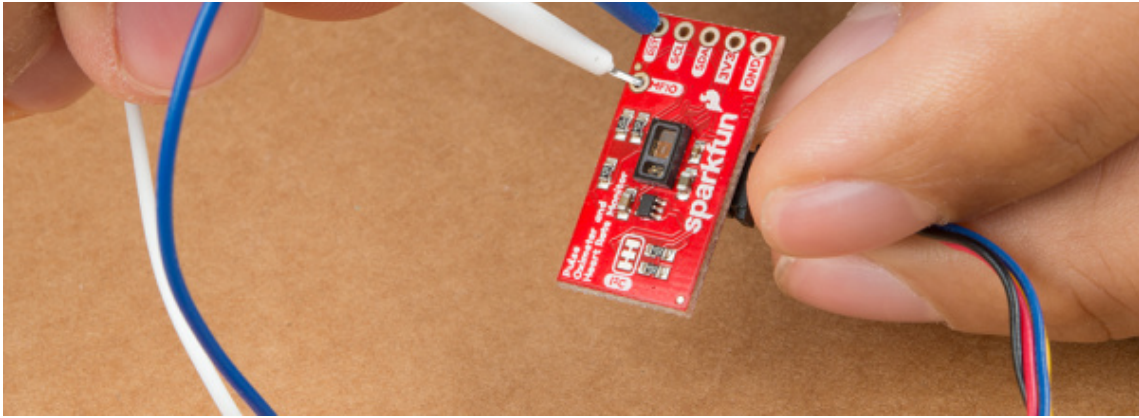
This board is an I²C based board and so we've included a Qwiic Connector. However, it isn't a "pure" Qwiic board as it requires two additional pins to be attached in order for it to function. However, you still don't need to solder if you have some of our [IC hook with pigtail](#) and in fact that's what I'll use for the following example. First, let's plug in our Qwiic Connector cable to either of the Qwiic connectors on the SparkFun Pulse Oximeter and Heart Rate Monitor.

Next we'll take the IC hooks and plug them into Redboard Qwiic into pins `5` and `4`.

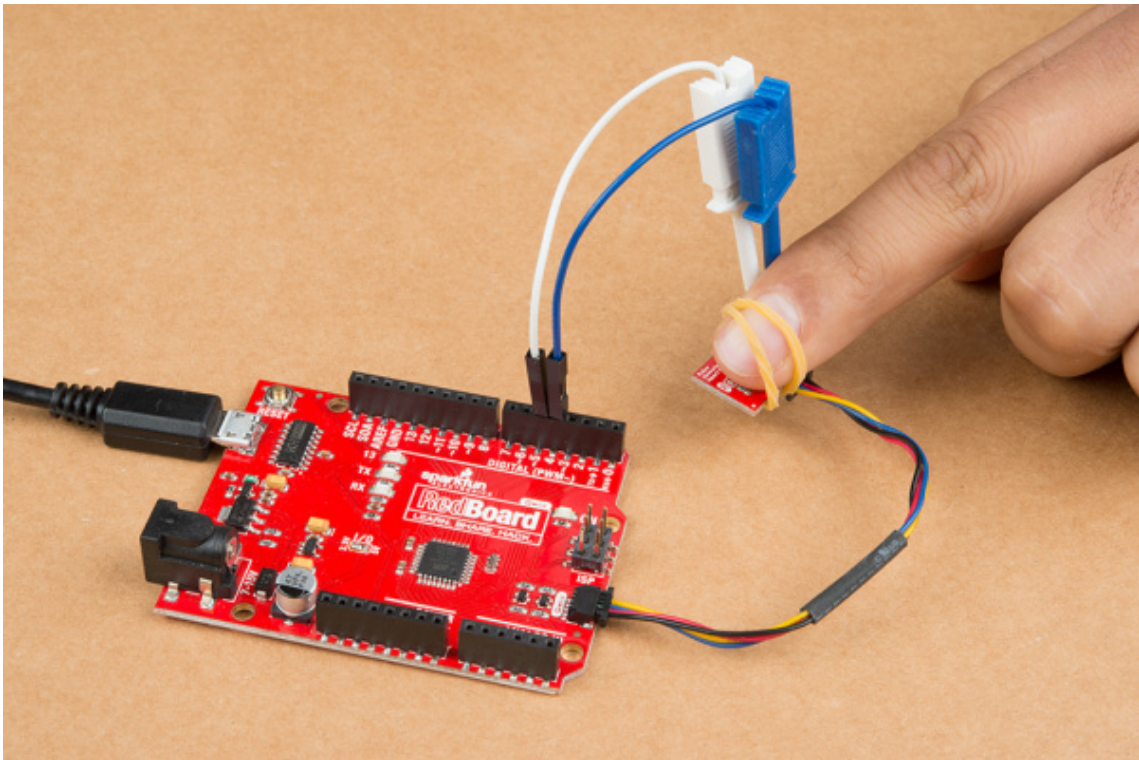


We'll then attach the claw side of the the IC hooks, the first in pin `5` to `MFIO` and the second plugged into pin `4` to `RESET`.





When we get to sensing your pulse and blood oxygen levels, it's important that when you place your finger onto the sensor that you place it lightly and with consistent pressure. You can try to do this without any support, but I found that a rubber band is a good place to start.



SparkFun Bio Sensor Arduino Library

Note: This example below assumes you are using the latest version of the Arduino IDE on your desktop. If this is your first time using Arduino, please review our tutorial on [installing the Arduino IDE](#). If you have not previously installed an Arduino library, please check out our [installation guide](#).

If you're using the RedBoard Qwiic and have never connected a CH340 device to your computer before, you may need to install drivers for the USB-to-serial converter. Check out our section on [How to Install CH340 Drivers](#) for help with the installation

We've written an Arduino library to make it even easier to get started with the SparkFun Pulse Oximeter and Heart Rate Monitor. The library will give you the full functionality of the sensor and provides example code to get the most out of your project. You can obtain these libraries through the Arduino Library Manager by searching **SparkFun Bio Sensor Arduino Library**. The second option is to download the ZIP file below from its [GitHub repository](#) to manually install it.

Reference Tables and Sensor Settings

This section shows some of the sensor's settings in clear tables so that you don't have to run to the datasheet for reference. It will also help to expand upon some unique characteristics of the SparkFun Bio Sensor Hub Library. Feel free to move beyond this section and jump to the **Example Section** below until this information becomes relevant in one of the examples.

BioData Information

As you'll see below, the library uses a *type* that is unique to the SparkFun Pulse Oximeter and Heart Rate Monitor. The name of this *type* is `bioData` and with it we'll be able to get at the biometric data that comes out of the board. Below is a table that shows all of the possible data stored within this mysterious new container, it has been named **body** for the table below and the following examples.

bioData Members	Information Provided
body.heartrate	Heart rate
body.oxygen	Blood Oxygen Levels
body.confidence	The Sensor's confidence in the reported data.
body.status	Finger Detection
body.irLed	Number of IR LED Samples
body.redLED	Number of Red LED Samples

One last thing! Not all of this data is available all the time. It will depend on if you configure the sensor for retrieving *just* **biometric** data, **sensor** data, or **both**!

BioData Mode 2

In addition to the information above, mode 2 also gives the following two data points.

bioData Members	Information Provided
body.rValue	SpO2 r Value
body.extStatus	Extended Finger Status Message

Finger Status

Below is a reference table for the `body.status` member which tells you if the sensor has detected a finger or some other object that is not a finger. It relays this information with four numbers: 0-3.

Status Number	Description
0	No Object Detected

1	Object Detected
2	Object Other Than Finger Detected
3	Finger Detected

Below is a reference table for the `body.exStatus` member which is an expansion of the first finger status messaging. This is enabled in mode 2 and contains 8 different values.

Status Number	Description
0	Success
1	Not Ready
-1	Object Detected
-2	Excessive Sensor Device Motion
-3	No object detected
-4	Pressing too hard
-5	Object other than finger detected
-6	Excessive finger motion

Pulse Width vs. Sample Collection

There is trade off between higher resolution (i.e. longer pulse width) and the number of samples that you can collect per second. The table below shows how the resolution and sample rate interact.

Samples/Second	Pulse Width (uS)			
	69	118	215	411
50	O	O	O	O
100	O	O	O	O
200	O	O	O	O
400	O	O	O	O
800	O	O	O	
1000	O	O		
1600	O			
Resolution (bits)	15	16	17	18

Example 1: Config BPM Mode 1

In this first example, we'll read the heart rate and blood oxygen level of the person we're monitoring. We'll also look at two other important values that the SparkFun Pulse Oximeter and Heart Rate Monitor provides so that you can ascertain whether the heart rate is accurate and whether a finger is being detected. Open the example up by heading to **File > Examples > SparkFun Bio Sensor Hub Library >**

Example1_config_BPM_Mode1.ino .

Let's start at the top of **Example 1: Config BPM Mode 1**. Of note here, is that when we create an instance of the library called `bioHub`, we provide the SparkFun Pulse Oximeter's address but also the pin numbers used on the Arduino that the `RESET` and `MFIO` are attached to: pin `4` and `5` respectively. These pins are necessary for the board's function, so make sure they're included here and also put in the correct order:

`RESET` then `MFIO` pin.

```
#include <SparkFun_Bio_Sensor_Hub_Library.h>
#include <Wire.h>

// No other Address options.
#define DEF_ADDR 0x55

// Reset pin, MFIO pin
const int resPin = 4;
const int mfioPin = 5;

// Takes address, reset pin, and MFIO pin.
SparkFun_Bio_Sensor_Hub bioHub(resPin, mfioPin);

bioData body;
```

Copy Code

Just above you'll see this funky *type* called `bioData`. This is a *type* that is unique to the SparkFun Pulse Oximeter and Heart Rate Monitor and it holds all the Biometric data of the sensor: Heart rate, confidence, blood oxygen levels, finger detection, led data, etc. I've provided a table just above that describes all the available information that it holds (see **Reference Tables and Sensor Settings**). Later in the example, we'll see how it's used.

Next let's look at the setup. There are two functions to point out. First, the `bioHub.begin()` function call makes sure that we can communicate with the sensor. Secondly and equally as important `bioHub.configBPM(MODE_ONE)`, configures the SparkFun Pulse Oximeter's settings and enables all of the necessary algorithms within the sensor to begin collecting data. Which data is collected depends on how the sensor is configured. You'll get biometric data with `bioHub.configBPM()`, you can just get LED data with `bioHub.configSensor()`, or you can get all the data with `bioHub.configSensorBPM()`. As soon as this is called, the sensor will begin collecting data. However, the sensor lags a couple of seconds behind when it begins sensing the data and when it actually gives that data to the user. I've put a **four** second delay at the end of setup to give some time for the data to catch up.

```
void setup(){

  Serial.begin(115200);

  Wire.begin();
  int result = bioHub.begin();
  if (!result)
    Serial.println("Sensor started!");
  else
    Serial.println("Could not communicate with the sensor!!!");

  Serial.println("Configuring Sensor....");
```

Copy Code


```
int error = bioHub.configBpm(MODE_ONE); // Configuring just the BPM settings.
if(!error){
  Serial.println("Sensor configured.");
}
else {
  Serial.println("Error configuring sensor.");
}
```

In the main loop, the biometric data is collected from the SparkFun Pulse Oximeter and Heart Rate Monitor with the function `bioHub.readBpm()`, and it's saved to `body`. Now to get at that information, we call `body.heartRate`, `body.oxygen`, etc. Easy!

```
void loop(){

  // Information from the readBpm function will be saved to our "body"
  // variable.
  body = bioHub.readBpm();
  Serial.print("HeartRate: ");
  Serial.println(body.heartRate);
  Serial.print("Confidence: ");
  Serial.println(body.confidence);
  Serial.print("Oxygen: ");
  Serial.println(body.oxygen);
  Serial.print("Status: ");
  Serial.println(body.status);
  delay(250); // Slowing it down, we don't need to break our necks here.
}
```

Copy Code

A note on `body.confidence` and `body.status`. The confidence level is the sensor's confidence in the heart rate that was reported. The status is whether or not the sensor has detected a finger. See the table above for the four possible status numbers and what they mean.

Example 2: Config BPM Mode 2

As opposed to Example 1, In Example 2's setup, we give the argument `MODE_TWO` to `bioHub.configBPM()` to get more information from the SparkFun Pulse Oximeter and Heart Rate Monitor. Specifically we'll get an *extended* finger status and the R value of the blood oxygen data.

```
Serial.println("Configuring Sensor....");
int error = bioHub.configBpm(MODE_TWO); // Configuring just the BPM settings.
if(!error){
  Serial.println("Sensor configured.");
}
else {
  Serial.println("Error configuring sensor.");
  Serial.print("Error: ");
  Serial.println(error);
}
```

Copy Code

Simple right? Nothing else changes except that when we pull the data we now have access to more data: `body.extStatus` and `body.rValue`.

```
void loop(){

  // Information from the readBpm function will be saved to our "body"
```

Copy Code

```
// variable.
body = bioHub.readBpm();
Serial.print("HeartRate: ");
Serial.println(body.heartRate);
Serial.print("Confidence: ");
Serial.println(body.confidence);
Serial.print("Oxygen: ");
Serial.println(body.oxygen);
Serial.print("Status: ");
Serial.println(body.status);
Serial.print("Extended Status: ");
Serial.println(body.extStatus);
Serial.print("Blood Oxygen R value: ");
Serial.println(body.rValue);
delay(250); // Slowing it down, we don't need to break our necks here.
```

Check the [reference table](#) above under *Finger Status* for more information on what each number means, there are eight. The R value refers to a [correlation coefficient](#) used to determine a statistical relationship between two variables: blood oxygen and a optical plate placed over the sensor. This does not refer to *the* glass shield on the sensor but rather a shield that would be placed over the sensor if you decided to implement this into a final product. In other words the company that manufactures this IC, Maxim Integrated has given the user a way to get faster FDA approval when using this IC in a final product. You can read more about that [here](#). For those of us just tinkering in a project it has no **Real** value. Get it?

Example 3: AGC Settings

I won't break down this example code because you have all the necessary tools to get you started with the SparkFun Pulse Oximeter and Heart Rate Monitor. However, there are a few more settings to fine tune the **Automatic Gain Control (AGC)** algorithm that the MAX32664 Sensor uses to automatically adjust the MAX30101 on the fly. This particular algorithm is being used in the first example because it's turned on automatically with the `configBPM()` function call. Configuring the sensor to give both sensor and biometric data (`configSensorBpm()`) does not have this algorithm enabled and so relies on the default settings of the MAX30101 sensor, unless of course you have configured the pulse width and sample collection yourself. In Example 4 we talk about how to modify these values.

Example 4: Adjust LED Values

The fourth example will show you how to adjust the accuracy of the SparkFun Pulse Oximeter and Heart Rate Monitor. We'll do this by adjusting the length of time that the LEDs inside the MAX30101 pulse, which will also impact how many samples we can get at a time. So we'll talk about these two settings and how they play against each other. Open the example up by heading to **File > Examples > SparkFun Bio Sensor Hub Library > Example4_config_LEDs_BPM.ino** .

Starting at the top, we assign the reset and mfio pins to pin `4` and `5` respectively. Below that we have two variables that will store the pulse width and the sample rate: `width` and `samples` . A longer pulse width changes the amount of time that the sensor's LEDs shine into the finger before ascertaining how much light was absorbed. This results in higher resolution data as the finger is fully illuminated before collecting data. However, the trade off is the sensor has less time to collect samples. For each increasing pulse width setting, there is a decrease in the amount of samples that can be collected. Check the table above under **Pulse Width vs Sample Collection** above to see all possible interactions.

```
const int resPin = 4;
const int mfioPin = 5;
```

Copy Code

```
// Possible widths: 69, 118, 215, 411us
int width = 411;
// Possible samples: 50, 100, 200, 400, 800, 1000, 1600, 3200 samples/second
// Not every sample amount is possible with every width; check out our hookup
// guide for more information.
int samples = 400;
int pulseWidthVal;
int sampleVal;

// Takes address, reset pin, and MFIO pin.
SparkFun_Bio_Sensor_Hub bioHub(resPin, mfioPin);

bioData body;
```

I'll reiterate what's stated in the first example. Just above you'll see this funky *type* called `bioData`. This is a *type* that is unique to the SparkFun Pulse Oximeter and Heart Rate Monitor and it holds all the Biometric data of the sensor: Heart rate, confidence, blood oxygen levels, finger detection, led data, etc. There is a table above under **Reference Tables and Sensor Settings** that displays the information available in `bioData`.

Unlike our first example, in this one we're we are calling `bioHub.configSensorBpm(MODE_ONE)` which tells the MAX32664 to give us both LED data as well as biometric data and to load it up into `bioData`. We'll see this come into play later on in the `loop` below.

```
int error = bioHub.configSensorBpm(MODE_ONE); // Configure Sensor and BPM mode
if(!error){
    Serial.println("Sensor configured.");
}
else {
    Serial.println("Error configuring sensor.");
    Serial.print("Error: ");
    Serial.println(error);
}
```

Copy Code

To set the pulse width, there is a call to `bioHub.setPulseWidth(width)`, giving it the variable `width` that we defined above that holds the value **411µS**. We then set the sample rate with `bioHub.setSampleRate(samples)`, again using the variable defined above. If you were to set a sample rate above what is capable at a particular pulse width, the sensor will automatically set it to the highest possible setting at that rate automatically. After configuring both settings, both values are read back with calls to `bioHub.readPulseWidth()` and `bioHub.readSampleRate()`.

```
error = bioHub.setPulseWidth(width);
if (!error){
    Serial.println("Pulse Width Set.");
}
else {
    Serial.println("Could not set Pulse Width.");
    Serial.print("Error: ");
    Serial.println(error);
}

// Check that the pulse width was set.
pulseWidthVal = bioHub.readPulseWidth();
Serial.print("Pulse Width: ");
Serial.println(pulseWidthVal);
```

Copy Code

```
// Set sample rate per second. Remember that not every sample rate is
// available with every pulse width. Check hookup guide for more information.
error = bioHub.setSampleRate(samples);
```

Here in the loop, we have a bit more information being printed out to the serial monitor than in the first example. Specifically there are the `body.irLed` and `body.redLed` data points which give us the number of light samples collected by the sensor for the respective LEDs.

```
void loop(){

  // Information from the readSensor function will be saved to our "body"
  // variable.
  body = bioHub.readSensorBpm();
  Serial.print("Infrared LED counts: ");
  Serial.println(body.irLed);
  Serial.print("Red LED counts: ");
  Serial.println(body.redLed);
  Serial.print("Heart rate: ");
  Serial.println(body.heartRate);
  Serial.print("Confidence: ");
  Serial.println(body.confidence);
  Serial.print("Blood Oxygen: ");
  Serial.println(body.oxygen);
  Serial.print("Status: ");
  Serial.println(body.status);
  delay(500); // Slowing it down, we don't need to break our necks here.
```

Copy Code

Troubleshooting

Heart Rate is Non-Existent!

It can't be reiterated enough, the sensor needs light but consistent pressure on the full plate of the sensor. I've also found that it's much easier to get readings from a warm hand then a cold one. If you continue to have issues check the finger status and extended finger status to get a hint at what the sensor is seeing. Check the [reference table](#) and scroll down to **Finger Status** to see all of the various error messages that are given by these status messages.

Error Configuring Sensor

First this products is not "pure" Qwiic because it requires that the `reset` and `mfio` pin be connected to your micro-controller. Check the [hardware hookup](#) section above for explicit instructions on how to do this.

Error Table

If you continue to have trouble configuring your sensor after checking that you've correctly hooked it up, then perhaps an error message can narrow it down for you. Below is a list of most of the error values from the datasheet. The final error message of **incorrect paramater** was implemented in the library to convey that an incorrect argument was used for any particular function.

Error	Description
0x01	ERROR UNAVAILABLE COMMAND
0x02	ERROR UNAVAILABLE FUNCTION

0x03	ERROR DATA FORMAT
0x04	ERROR INPUT VALUE
0x05	ERROR TRY AGAIN
0xFF (255)	ERROR UNKNOWN
0xEE (238)	INCORRECT PARAMETER

Resources and Going Further

For more on the AS3935, check out the links below:

- [Schematic \(PDF\)](#)
- [Eagle Files \(ZIP\)](#)
- [Board Dimensions \(PNG\)](#)
- Datasheets (PDF)
 - [MAX30101 \(PDF\)](#)
 - [MAX32664 \(PDF\)](#)
- [MAX32664 User Guide \(PDF\)](#)
- GitHub
 - [Arduino Library](#)
 - [Product Repo](#) - Design files and more datasheets!
- [SFE Product Showcase](#)

Need some other [sensor to measure biometrics](#) for your project? Check out some of the ones listed below.



SparkFun Air Quality PM1/PM2.5/PM10 Sensor - BMV080 (Qwiic)

SEN-26554

\$64.95



MyoWare 2 Muscle Sensor Development Kit

KIT-27920

\$154.95



SparkFun Qwiic MicroPressure Sensor

SEN-16476

\$37.95



SparkFun Pulse Oximeter and Heart Rate Sensor - MAX30101 & MAX32664 (Qwiic)

SEN-15219

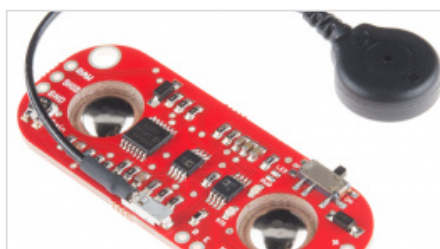
\$48.95

Need some inspiration for your next project? Check out some of these related tutorials to sense your environment!



Hackers in Residence - Hacking MindWave Mobile

Review, teardown, and hacking tutorial for the MindWave Mobile, a \$99 commercial grade EEG sensor.



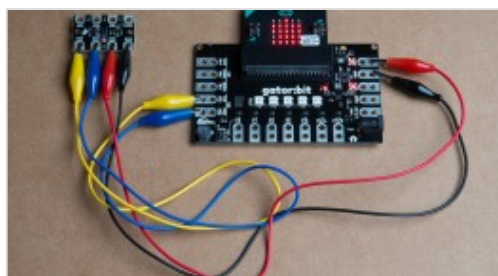
MyoWare Muscle Sensor Kit

Line of products to work with the MyoWare muscle sensor from Advancer Technologies.



Getting Started with Walabot

See through walls, track objects, monitor breathing patterns, and more using the power of radio frequency with the Walabot! In this tutorial, we will explore Walabot's features using the Software Demo Kit (SDK) on Windows and the Application Programming Interface (API) on Linux-based distributions for embedded projects.



SparkFun gator:particle Hookup Guide

The gator:particle is an I2C heart-rate monitor and pulse oximeter that can be used as a particle sensor. This tutorial will get you started using the gator:particle with the micro:bit platform.



ABOUT SPARKFUN

[Read Our Story](#)

[Press & Media](#)

[SparkFun Education](#) 

[Job Openings](#)

PARTNER WITH US

[See Our Partners](#)

[View Our Services](#)

[Become a Distributor/Reseller](#)

[Receive Volume Discounts](#)

SUPPORT

[Customer Support](#)

[Purchase Order & Payment Terms](#)

[Technical Assistance](#)

[FAQs](#)

SITE INFORMATION

[Terms Of Service](#)

[Privacy Policy](#)

[Compliance](#)

[Site Map](#)

SUBSCRIBE

Sign up to receive weekly product releases, special offers and more.

First Name:

Email: 

SUBSCRIBE

VIEW OUR SOCIAL CHANNELS



SparkFun Electronics®

6333 Dry Creek Parkway,
Niwot, CO 80503